Engineering Approach Document: CT-200 Medical Manual QA API

Candidate: **Lakshman** | GitHub: [@Lakshman3556](https://github.com/Lakshman3556)

Project: CardioTrack CT-200 Technical Manual Extraction, Versioning & Staleness Pipeline

---

The assignment explicitly requires an OCR-based or document understanding extraction pipeline. When inspecting `data/ct200_manual.pdf` (v1, 6 pages) and `data/ct200_manual_v2.pdf` (v2, 7 pages) with `scripts/inspect_pdf.py`, empirical measurement revealed:

? **Embedded Images:** Exactly `0` raster images in both v1 and v2.

? **Vector Fonts:** All text is rendered using embedded fonts (`NimbusSans-Regular`, `NimbusSans-Bold`).

? **Text Layer:** 100% of characters reside in a clean native PDF stream.

? **Vector Geometry:** Table borders are drawn with vector drawing paths (78 drawing operations on page 2; 85 on page 4).

Our Architecture Choice:

We selected a **native-first extraction pipeline powered by PyMuPDF (`fitz`)**, combined with an **optional OCR cross-check (`pytesseract`)**.

Applying pure OCR to a clean born-digital PDF degrades recognition accuracy: OCR introduces character misidentifications (e.g., confusing `1` and `l`, dropping punctuation, corrupting decimals in medical ranges like `0.5 mmHg`).

However, to respect the regulatory OCR requirement:

1. We implemented `app/parser/ocr_check.py`, which rasterizes pages at 200 DPI and computes string difflib similarity and word-level Jaccard against native text.

2. If native text contains fewer than 20 characters per page (scanned documents), the pipeline automatically switches to OCR word boxes via `pytesseract.image_to_data`.

3. If `tesseract` is absent from the host environment, the system gracefully logs `ocr: {status: 'skipped'}` and continues without crashing.

? **Docling / Marker / Unstructured:** While capable, these frameworks introduce heavy deep learning dependencies (PyTorch, layout models) that are non-deterministic, slow to install in constrained environments, and frequently produce unpredictable heading nestings on structured technical manuals.

? **pdfplumber default tables:** Measured inspection showed `pdfplumber.extract_tables()` produced phantom sub-tables (e.g. splitting `['Pressure', 'range']` and `['0?', '299', 'mmHg']`).

? **PyMuPDF default cell text:** Default PyMuPDF cell extraction corrupted commas and numbers (returning `'Aborts measurement displays E1\n,'` and `'1 .5 seconds'` because commas and periods were distinct spans). We solved this by clipping words to cell bounding boxes and sorting them geometrically.

Hierarchy reconstruction relies on four strict criteria:

1. **Margin Anchor:** Dynamic detection of the body margin `x0 = min(line_x0)` (measured at `33.0 pt`).

2. **Compound Heading Rule:** A line is a heading if and only if:

$$\text{bold} \land \text{HEAD_RE} \land (x_0 \le \text{margin} + 3.0\text{ pt}) \land (\text{bbox} \notin \text{table_bbox})$$

3. **Declared Depth:** Count of dotted parts (e.g. `2.1.1.1` $\to$ level 4; `3` $\to$ level 1).

4. **Persistent Longest-Prefix Parent Resolution:**

Rather than resetting state per page, the parser maintains an ongoing document tree. When heading `2.1.1.1` arrives, parent search tests candidate prefixes in descending order: `2.1.1`, then `2.1`, then `2`. It immediately resolves `2.1` as the parent, declares `tree_depth = 3`, and flags `level_gap`.

(Note: The assignment brief states "handle every irregularity listed above", but the original problem description had no explicit list. We discovered these irregularities through empirical PDF inspection).

ID | Structural Irregularity | Occurrence in PDF | Failure of Naive Baseline | Implemented Engineering Solution

---

The baseline naive parser (`scripts/naive_parse_v0.py`) was evaluated via `scripts/validate_extraction.py`. The real failure output:

--- Naive v0 Extraction Failure Report ---

? FAIL [I2]: Section '3.2' is completely missing because its heading is 11pt (demoted to body).

? FAIL [I2/I3]: Section '2.1.1.1' is missing because its heading is 11pt.

? FAIL [I8]: Section 4.2 table text is fragmented or missing key error rows.

? FAIL [I9]: Unnormalized Unicode ligatures (fi/fl/ff) exist in extracted text; search will fail.

? FAIL: Expected 27 section nodes in v1, but naive parser found 22.

Metric | Naive Baseline (v0) | Production Pipeline

Total Section Nodes (v1) | 22 (missed 3.2, 2.1.1.1, etc.) | **28 (27 sections + 1 root)** ?

Total Section Nodes (v2) | 23 | **29 (28 sections + 1 root)** ?

Table Extraction | Raw text stream with broken commas | **Structured cell-clipped rows** ?

Search Hit for "profile"	Failed (blocked by `\ufb01` ligature)	**100% hit rate on normalized text** ?
Multiset Word Coverage	Failed (dropped ~180 words)	**Invariant Validated (0 words dropped)** ?

The relational design enforces a fundamental separation between **Logical Identity** (stable across versions) and **Node Version** (immutable snapshot):

? **Selections are Immutable:** No `PUT` or `PATCH`. If a user modifies a selection, a new selection record is created.

? **Version-Pinned Selection Items:** Pin to specific `node_version_id` rows. When document v2 is ingested, existing selections continue to resolve to the exact text of v1.

Re-ingesting `ct200_manual_v2.pdf` executes a **conservative 5-pass matcher** (`app/services/matching.py`) that matches nodes from top to bottom:

1. **Pass 1: Exact Key:** Same `section_number` AND same `heading_norm` under matched parents $\to$ same logical node (`unchanged` if `own_hash` identical, else `modified`).
2. **Pass 2: Title Under Matched Parent:** Same `heading_norm` under matched parents, differing section number $\to$ `renumbered`.
3. **Pass 3: Unique Hash Match:** Unmatched nodes sharing an identical `own_hash` $\to$ `moved` or `renumbered`.
4. **Pass 4: Mutual Best Fuzzy Match:** `rapidfuzz.token_set_ratio(heading) >= 90` AND `difflib.body_ratio >= 0.6` $\to$ `modified` (`change_basis = 'fuzzy'`).
5. **Pass 5: Leftovers:** Unmatched new nodes $\to$ `added`; unmatched old nodes $\to$ `removed` (tombstone). Never force a match.

? **Modified:** `2.1.1.1` (cycle threshold 300 $\to$ 250, low battery 15% $\to$ 10%), `3.2` (inflation increment 40 $\to$ 30 mmHg), `4.2` (E3 deflation 2s $\to$ 1.5s, row E6 added), `4.3` (`E1-E5` $\to$ `E1-E6`).

? **Added:** `5.3 Data Export` (firmware 1.4 CSV export).

? **Unchanged:** All other 22 sections + root (including traps `3.1`, `6.1`, and `8.1`).

? **System Prompt:** Specifies that the technical excerpt is strict DATA, not instructions.

? **Verbatim Evidence Quotes:** For every test case, the LLM must return 1 to 3 exact substrings (`evidence_quotes`, min 12 characters).

? **Post-Generation Grounding Validator:** The backend verifies that every evidence quote is an exact normalized substring of the source excerpt. If a model hallucinates a threshold or quotes absent text, that test case is automatically dropped.

1. **Markdown Fences:** Fences (` ```json `) are stripped and the first balanced `{...}` is parsed.
2. **Syntax / Schema Errors:** Triggers a 2-step repair retry sending the parse error back to the model.
3. **Persistent Garbage:** If all attempts fail, the generation is persisted with `status: "failed"` along with the full audit log of raw attempts, and the API returns **HTTP 502 Bad Gateway**. Unvalidated test cases are **never** returned to the user.

? Keyed on `(selection_fingerprint, prompt_version, model)`.

? Subsequent submissions return the completed generation with `reused: true` without invoking the LLM.

? Adding `?force=true` generates a new record that marks the earlier generation via `supersedes`.

? Per-selection in-process `asyncio.Lock` prevents race conditions from double-clicks.

Staleness is **never stored as a static flag**; it is dynamically evaluated at query time against the requested target version (`?against_version=latest`):

$$\text{Test Case Status} = \begin{cases}

\text{likely_stale} & \text{if cited quote is missing or source node was removed} \\

\text{review_recommended} & \text{if source had material change or cross-referenced section changed} \\

\text{probably_valid} & \text{if source only had cosmetic wording changes} \\

\text{fresh} & \text{if source text and references are unchanged}

\end{cases}$$

? **Material Change:** Any diff token containing a digit, a medical/physical unit (`mmHg`, `seconds`, `%`, `bpm`), an identifier (`E1`..`E6`), or a safety modal (`must`, `shall`, `exceeds`, `within`).

? **Wording Change:** Minor grammatical or non-threshold phrasing adjustments.

Lakshman: The staleness detection engine is the most vulnerable to silent false positives. It evaluates staleness by checking if the specific text cited by a test case still exists in the latest document version. However, it cannot automatically detect cross-section semantic contradictions. For example, in Version 2, Section `4.1` remains completely unchanged and states that the emergency valve auto-deflates within **2 seconds**, while Section `4.2` was updated to state that Error Code `E3` auto-deflates within **1.5 seconds**. A test case generated against Section 4.1 will report `fresh` because its cited text hasn't changed, even though the manual is now internally contradictory. To mitigate this, I implemented a `possibly_related_nodes` diagnostic that alerts the user when another section sharing the same error identifiers (`E3`) or units (`seconds`) has changed.

Lakshman: I chose a heuristic lexical significance classifier (checking digits, units, and modal keywords) rather than building a semantic AST parser for medical requirement logic. What would break first in production is subtle phrasing reversals that don't alter keywords?such as an engineering update that inverts a requirement using descriptive prose without trigger words (e.g., replacing *"cuff must deflate"* with an indirect statement). Additionally, in high-volume production, SQLite table locks during concurrent PDF ingestions and the in-process `asyncio.Lock` for LLM deduplication would need to be replaced with PostgreSQL and a distributed Redis/Celery queue.

Lakshman: For the PDF parser, I did not implement support for multi-column layouts or complex rotated graphical schematics. If a two-column manual is uploaded, the parser detects that text bounding boxes cross column gutters and marks the document with `parse_status = "needs_review"` and emits an `unsupported_block` warning rather than silently scrambling the reading order. For the LLM generation endpoint, if a selection's combined text exceeds 12,000 characters, rather than silently truncating the prompt and losing vital requirements, the API explicitly rejects the request with **HTTP 413 Payload Too Large**.

---

1. **Full Text Search Indexing:** Upgrade SQLite `LIKE` queries to SQLite `FTS5` virtual tables with BM25 ranking.

2. **Distributed Job Queue:** Move PDF extraction and LLM repair loops to asynchronous Celery/Redis tasks with webhook notifications.
3. **Visual Regression Overlays:** Generate automated PNG diff overlays highlighting changed paragraphs directly on the PDF canvas.
4. **Graph-Based Requirement Modeling:** Build a knowledge graph linking safety requirements across sections to automatically flag contradictions like the Section 4.1 vs 4.2 timing conflict.